گفتگوی کامل پروژه گوهر ولا

بک‌اند Django/Wagtail و فرانت‌اند Next.js

Total messages: 86

کاربر #1

This session is being continued from a previous conversation that ran out of context. The summary below covers the earlier portion of the conversation.

Summary:
1. Primary Request and Intent:
The user owns a Persian jewelry e-commerce site called "گوهر ولا" (Goharevela) at goharevela.ir. Over the session:
- Renamed brand from "حجره شوشتری" to "گوهر ولا" everywhere in code
- Changed card holder name in checkout to "علی اصغر کیانی"
- Set up git repo on server, created .gitignore, removed db.sqlite3 from entire git history (force-pushed)
- Created `deploy.sh` for server deployment (systemd only, no PM2/Docker)
- Created CLAUDE.md documenting deployment setup
- Made the home page fully editable via Wagtail StreamField (hero slider, trust bar, categories, product sections, special offer, features, blog preview, newsletter)
- Made header, footer, and menus editable via Wagtail settings+snippets (SiteSettings + Menu/MenuItem)
- Tried to import products from eitaa.com (blocked by network policy — explained 3 options)
- **LATEST/PENDING:** User wants infrastructure for payment gateway, SMS verification, and shipping — added without needing code changes later

2. Key Technical Concepts:
- Django 4.2 + DRF + Wagtail 6.1 (StreamField, BaseGenericSetting, register_snippet, Orderable)
- Next.js 14 App Router with server components and `force-dynamic`
- Wagtail StreamField blocks for CMS-editable page sections
- `wagtail.contrib.settings` for global site settings
- ClusterableModel + ParentalKey + Orderable for Menu/MenuItem
- JWT authentication (SimpleJWT)
- Zustand for client-state (cart, auth)
- TailwindCSS RTL layout
- systemd for both frontend (goharevela-frontend) and backend (gunicorn)
- git filter-branch + gc to purge sensitive data from history
- Management commands: seed_homepage, setup_homepage, seed_site
- Fallback pattern: CMS data → hardcoded defaults (site never breaks)

3. Files and Code Sections:

- **`deploy.sh`** (root): Production deploy script

- git pull → pip install → migrate → collectstatic → systemctl restart gunicorn → npm install → npm run
systemctl restart goharevela-frontend
- Variables: REPO_DIR, BRANCH, FRONTEND_SERVICE (default: goharevela-frontend),
BACKEND_SERVICE (default: gunicorn), VENV_DIR, DJANGO_SETTINGS
- Usage: `./deploy.sh` or `FRONTEND_SERVICE=name BACKEND_SERVICE=name ./deploy.sh`

- **`CLAUDE.md`** (root): Documents that production uses systemd ONLY (not Docker, not PM2),
frontend=goharevela-frontend, backend=gunicorn, db/env/media never in git

- **`.gitignore`** (root): Ignores `*.sqlite3`, `__pycache__/`, `*.pyc`, `.env`, `node_modules/`, `.next/`,
`media/`, `staticfiles/`, `*.tsbuildinfo`

;backend/config/settings/base.py`**: Added `'wagtail.contrib.settings'` to INSTALLED_APPS`** -
'■■ ■■■■' = WAGTAIL_SITE_NAME

- **`backend/config/urls.py`**: Added `path('api/site/', include('apps.cms.urls', namespace='site'))`

- **`backend/apps/cms/blocks.py`** (new): Defines HOME_BLOCKS StreamField:
- HeroSliderBlock (slides: title/subtitle/badge/cta_text/cta_link/accent)
- TrustBarBlock (items: icon/title/description)
- CategoriesSectionBlock (heading/subheading)
- ProductSectionBlock (source: new|bestsellers|featured, heading/subheading/link_text/link_url)
- SpecialOfferBlock (badge/title/description/button_text/button_link)
- FeaturesBlock (heading/subheading/items: icon/title/description)
- BlogPreviewBlock (heading/subheading/link_text)
- NewsletterBlock (title/description/button_text)

- **`backend/apps/cms/models.py`**:
- Replaced `SiteConfiguration` with `SiteSettings(BaseGenericSetting)` with fields: brand_name,
brand_latin, brand_description, phone_primary, phone_secondary, email, address, working_hours,
instagram, telegram, whatsapp, aparat, topbar_message
- Added `Menu(ClusterableModel)` snippet with handle choices: main/footer_quick/footer_bottom
- Added `MenuItem(Orderable)` with menu FK, label, link, open_in_new_tab
- Added `body = StreamField(HOME_BLOCKS, blank=True, use_json_field=True)` to `HomePage`

- **`backend/apps/cms/views.py`** (new): `site_config` API view at GET /api/site/config/ returning
`{settings: {...}, menus: {main: [...], footer_quick: [...], footer_bottom: [...]}}`

- **`backend/apps/cms/urls.py`** (new): `path('config/', views.site_config, name='site-config')`

- **`backend/apps/cms/migrations/0002_homepage_body.py`**: Adds body StreamField to HomePage

- **`backend/apps/cms/migrations/0003_menu_menuitem_sitesettings_delete_siteconfiguration.py`**:

Creates Menu, MenuItem, SiteSettings; deletes SiteConfiguration

- **`backend/apps/cms/management/commands/seed_homepage.py`**: Populates HomePage.body with DEFAULT_BODY (all 9 sections). Usage: `python manage.py seed_homepage [--force]`

- **`backend/apps/cms/management/commands/setup_homepage.py`**: For servers with default Wagtail welcome page as root. Creates HomePage, moves blog under it, sets as site root, deletes old page, seeds sections. slug='home-gohar' (not 'home' — already taken). Idempotent. Usage: `python manage.py setup_homepage [--keep-old]`

- **`backend/apps/cms/management/commands/seed_site.py`**: Populates SiteSettings + 3 menus with current hardcoded values. Main menu built from actual categories in DB. Usage: `python manage.py seed_site [--force]`

- **`frontend/types/index.ts`**: Added HomeBlock union type (hero_slider|trust_bar|categories|products|special_offer|features|blog_preview|newsletter), HeroSlide, TrustItem, FeatureItem, HomePageData, SiteSettings, MenuLink, SiteConfig interfaces

- **`frontend/lib/api.ts`**: Added `getHomePage()` (GET /api/cms/pages/?type=cms.HomePage&fields=body), `getSiteConfig()` (GET /api/site/config/); added SiteConfig to imports

- **`frontend/lib/siteConfig.ts`** (new): Client-side cache for site config — `loadSiteConfig()` fetches once and caches the promise

- **`frontend/components/home/sections.tsx`** (new): Presentational components driven by CMS data: TrustBar, CategoriesSection, ProductSection, SpecialOffer, Features, BlogPreviewSection, Newsletter

- **`frontend/app/HeroSlider.tsx`**: Now accepts optional `slides?: HeroSlide[]` prop; falls back to DEFAULT_SLIDES if empty

- **`frontend/app/page.tsx`**: Rewritten — fetches `getHomePage()`, renders blocks via `renderBlock()` switch, falls back to hardcoded layout if body empty

- **`frontend/components/layout/Header.tsx`**: Reads phone, workingHours, topbarMessage, brandName, brandLatin from `loadSiteConfig()` with fallbacks

- **`frontend/components/layout/Navigation.tsx`**: Reads main menu from `loadSiteConfig()` with fallback to category-based default; `categorySlugFromLink()` extracts slug for dropdown

- **`frontend/components/layout/Footer.tsx`**: Reads all contact/social/brand from settings; quickLinks and bottomLinks from menus with fallbacks; copyright uses brandName

`[illegible] [illegible] [illegible] :[illegible] [illegible]` :frontend/app/checkout/page.tsx`** line 407`** -

4. Errors and fixes:
- **perl modified .pyc files**: Used `git checkout -- '*.pyc'` to restore
- **git push rejected (remote ahead)**: Used `git pull --rebase` then push
- **"dubious ownership" git error on server**: Add `git config --global --add safe.directory /var/www/goharevela`
- **slug 'home' already taken for HomePage**: Changed to slug='home-gogar' in setup_homepage.py
- **DB sqlite3 in git history**: Used `git filter-branch --force --index-filter 'git rm --cached --ignore-unmatch backend/db.sqlite3'` + purge refs/original + gc; then force-pushed both branches
- **npm install 403 from npmmirror.com**: Removed package-lock.json temporarily, installed with `--registry=https://registry.npmjs.org/`, then restored original lockfile
- **AskUserQuestion missing `question` field**: Kept failing — moved forward without asking

.in /cms/ — root was default Wagtail page "[illegible] [illegible]" No HomePage on server**: User saw no** -
Created `setup_homepage` command to fix
— "[illegible] pm2 [illegible]" PM2 assumption**: Initially wrote deploy.sh with PM2 for frontend. User said** -
.to systemd for both services. Updated CLAUDE.md accordingly

5. Problem Solving:
- **Editable homepage**: Entire homepage was hardcoded in Next.js. Solution: Wagtail StreamField blocks on HomePage model, API to expose them, frontend renderBlock() with safe fallbacks.
- **Editable header/footer/menus**: All contact info and navigation was hardcoded. Solution: SiteSettings (BaseGenericSetting) + Menu/MenuItem (snippets) + /api/site/config/ endpoint + loadSiteConfig() client cache.
- **Eitaa product import**: Network policy blocks eitaa.com. Explained 3 options: server-side import command, manual data entry, network access policy change. Pending user choice.
- **Payment/SMS/Shipping without code**: PENDING — this is the last request. Plan: create ProviderSettings model in Wagtail with toggle fields and API key fields; adapters check settings at runtime.

6. All user messages:

"[illegible] [illegible] [illegible] / [illegible] [illegible] [illegible] [illegible] [illegible] [illegible]" -
"[illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible]" -
[illegible] [illegible] / ...Stop hook feedback: [~/.claude/stop-hook-git-check.sh]: There are uncommitted changes" -
"[illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible]
"[illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible]" -
- [Screenshot: git fetch fatal: not a git repository]
[illegible] [illegible] [illegible] / [illegible] [illegible] pm2 [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible]" -
(correction: no PM2) "[illegible]
(stronger correction) "[illegible] pm2 [illegible] [illegible] [illegible] [illegible] [illegible]" -
- [Screenshot: systemctl list-units showing goharevela-frontend.service and gunicorn.service]
[illegible] [illegible] [illegible] [illegible] [illegible] / [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible]" -
[illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] / [illegible] [illegible] [illegible]

[redacted] Screenshot: Wagtail dashboard showing] -
[redacted]
[redacted] + [redacted] [Screenshot: Wagtail pages list] -
- [Screenshot showing SITE root => 'Welcome to your new Wagtail site!' | : Page]
[redacted] [/in /cms/pages/5 [redacted] Screenshot showing] -
[redacted] -
[redacted]
[redacted] -
[redacted] -
"https://eitaa.com/gohare_vela/126
[redacted] -
(LATEST REQUEST - PENDING) [redacted]

Security constraints (verbatim):
[redacted]
- Deploy is systemd ONLY — not Docker, not PM2

7. Pending Tasks:
[redacted] Payment/SMS/Shipping infrastructure without code**: Add provider settings to admin panel** -
[redacted]
(shipping methods with cost/threshold
- **Eitaa product import**: Blocked — user needs to choose from 3 options (server-side import command, manual entry, or network policy change)
- After deploy: run `python manage.py seed_site --settings=config.settings.production` and `python manage.py setup_homepage --settings=config.settings.production` on server

8. Current Work:
The last completed work was implementing editable header/footer/menus via Wagtail:
- `SiteSettings` (BaseGenericSetting) for brand/contact/social/topbar
- `Menu` + `MenuItem` snippets for main nav, footer quick links, footer bottom links
- `/api/site/config/` endpoint in `backend/apps/cms/views.py`
- `frontend/lib/siteConfig.ts` cached loader
- Header, Navigation, Footer all wired to CMS with fallbacks
- Management command `seed_site.py`
- All committed and pushed to `claude/clever-carson-r6f0sg`

The user's LATEST request (not yet started) is:
[redacted]
[redacted]

9. Optional Next Step:

[illegible]" :The next step is to implement the payment/SMS/shipping infrastructure as requested
[illegible]
"[illegible]

Plan:
1. Create `PaymentSettings`, `SmsSettings`, `ShippingMethod` models in `backend/apps/cms/models.p
using `BaseGenericSetting` (for payment/SMS — singleton configs with API key fields + enabled toggle
and a snippet/Orderable for `ShippingMethod` (name, price, free_above_amount, estimated_days,
enabled)
2. Write adapters in backend that check settings at runtime (ZarinPal, IDPay for payment; Kavenegar,
SMS.ir for SMS)
3. Wire to existing order/checkout views
4. Ask user which specific providers to support before writing adapters

If you need specific details from before compaction (like exact code snippets, error messages, or conte
you generated), read the full transcript at:
/root/.claude/projects/-home-user-goharevela/7a9e8c0e-b928-5c34-84bb-81990967a523.jsonl
Continue the conversation from where it left off without asking the user any further questions. Resume
directly — do not acknowledge the summary, do not recap what was happening, do not preface with "I'
continue" or similar. Pick up the last task as if the break never happened.

دستیار (2# (Claude

I'll start implementing the payment, SMS, and shipping infrastructure. Let me first explore the existing backend structure.

دستیار (3# (Claude

Let me read the key files I need to modify before implementing.

دستیار (4# (Claude

Now let me read the orders views, urls, and the checkout frontend page.

دستیار (5# (Claude

Let me read a few more files before implementing.

دستیار (6# (Claude

Let me read the orders serializers before writing the code.

دستیار (7# (Claude

Let me read the last migration file to understand the migration pattern for Wagtail settings.

دستیار (8# (Claude

Now I have the full picture. Let me implement everything in parallel — I'll write the backend models, adapters, views, and frontend changes simultaneously.

دستیار (Claude) #9

Now I have everything I need. I'll implement all the changes in parallel — backend models, adapters, views, frontend updates, and migration.

دستیار (Claude) #10

Now create the payment adapters file:

دستیار (Claude) #11

Now update the orders serializer to support `shipping_method_id`:

دستیار (Claude) #12

Now update the `create_order` method to use shipping method:

دستیار (Claude) #13

Now update `orders/views.py` to add the new views:

دستیار (Claude) #14

Now add the new views at the bottom of `orders/views.py`:

دستیار (Claude) #15

Now append the new views at the end of the file:

دستیار (Claude) #16

Now update `orders/urls.py`:

دستیار (Claude) #17

Now add OTP views to accounts:

دستیار (Claude) #18

You've hit your session limit · resets 4:10pm (UTC)

کاربر #19

یه امکاناتی اضافه کن که اگه خواستم درگاه پرداخت، تایید پیامکی و ارسال پستی رو اضافه
کنم نیازی به کد زدن نداشته باشم

دستیار (Claude) #20

You've hit your session limit · resets 4:10pm (UTC)

کاربر #21

یه امکاناتی اضافه کن که اگه خواستم درگاه پرداخت، تایید پیامکی و ارسال پستی رو اضافه
کنم نیازی به کد زدن نداشته باشم
یه امکاناتی اضافه کن که اگه خواستم درگاه پرداخت، تایید پیامکی و ارسال پستی رو اضافه
کنم نیازی به کد زدن نداشته باشم

دستیار (Claude) #22

I'm mid-implementation of exactly this. Let me continue finishing it. Adding the OTP request/verify views to the accounts app:

دستیار (Claude) #23

Now add the OTP URLs to accounts:

دستیار (Claude) #24

Now expose payment/shipping config in the site config API so the frontend can render the right options. Let me update `cms/views.py`:

دستیار (Claude) #25

Now let me check the production settings for FRONTEND_URL and CACHES (OTP uses cache):

دستیار (Claude) #26

Cache is available. Now let me add `FRONTEND_URL` to settings. Let me check base.py around relevant config:

دستیار (Claude) #27

Now check that `requests` is in requirements.txt — the adapters depend on it:

دستیار (Claude) #28

Now let me generate the migration. Let me try running makemigrations:

دستیار (Claude) #29

Migration created. Let me verify the system check passes and the migration is valid:

دستیار (Claude) #30

Now let me apply migrations to the dev DB to confirm they work, then wire up the frontend. Let me update the frontend types and API first:

دستیار (Claude) #31

Now update `OrderCreateData` to include `shipping_method_id`, and add API functions for payment initiation:

دستیار (Claude) #32

Now add API functions for payment initiation in `frontend/lib/api.ts`:

دستیار (Claude) #33

Now update the checkout page to: load shipping methods + payment config from site config, drive card-to-card details and shipping cost from CMS, and redirect to gateway for online payment. Let me update the checkout page:

دستیار (Claude) #34

I need to re-read the checkout file since state was reset:

دستیار (Claude) #35

Now I'll make all the checkout edits. First the imports and FormData:

دستیار (Claude) #36

Now add state, load config, and adjust submit logic:

دستیار (Claude) #37

Now update the shipping cost calculation and handleSubmit:

دستیار (Claude) #38

Now update the success state check (changed from step 4 to 5):

دستیار (Claude) #39

Now add the shipping method selection UI in Step 1 (after the notes field, before the "next" button), and update Step 2 to filter payment options + use CMS card details. First, Step 1 shipping selector:

دستیار (Claude) #40

Now update Step 2 to filter payment options by config and use CMS card details:

دستیار (Claude) #41

Now update the final submit button text to reflect online payment flow:

دستیار (Claude) #42

Now let me create a management command to seed default payment/SMS/shipping settings, and verify the migration applies. Let me check the existing seed_site command structure:

دستیار (Claude) #43

Now create a `seed_commerce` command to seed default shipping methods and payment card details:

دستیار (Claude) #44

Now let me apply the migration on the dev DB and run a quick check that everything imports/works:

دستیار (Claude) #45

Adapters fall back correctly when nothing is configured. Let me run the seed command and the full check, plus a frontend typecheck:

دستیار (Claude) #46

The `validate()` function still uses the old `Partial<FormData>` type. Let me fix it:

دستیار (Claude) #47

Typecheck passes. Now let me update the deployment docs in CLAUDE.md to mention the new

commands, and update deploy.sh if needed. Let me check deploy.sh:

دستیار (Claude #48)

Let me add documentation to CLAUDE.md about the new commerce features:

دستیار (Claude #49)

Now let me verify there are no Python import issues by running the test/check on the orders app fully, then commit. Let me confirm git branch and status:

دستیار (Claude #50)

Let me refine one thing: avoid double SMS for online payments (order-creation SMS + payment-success SMS). Only send on creation for card-to-card:

دستیار (Claude #51)

Now let me commit and push:

دستیار (Claude #52)

تمام شد. زیرساخت کامل پرداخت، پیامک و ارسال را اضافه کردم — حالا همه را **بدون کدزدن** از پنل `/cms/` می‌توانی فعال/مدیریت کنی.

چه چیزی اضافه شد

۱. درگاه پرداخت (تنظیمات «تنظیمات پرداخت»)
- روشن/خاموش کردن پرداخت آنلاین، انتخاب درگاه (**زرین‌پال** یا **آی‌دی‌پی**)، وارد کردن Merchant ID/API Key، حالت آزمایشی
- اطلاعات کارت به کارت (شماره کارت + نام صاحب کارت) — دیگر در کد hardcode نیست
- آداپتور درگاه در زمان اجرا بر اساس تنظیمات انتخاب می‌شود؛ تا وقتی کلید وارد نشده، فقط کارت به کارت نمایش داده می‌شود (fallback امن)
- مسیر کامل: شروع پرداخت ← انتقال به درگاه ← callback ← تأیید ← بروزرسانی وضعیت سفارش + پیامک

۲. تأیید پیامکی (تنظیمات «تنظیمات پیامک»)
- انتخاب سرویس (**کاوه‌نگار** یا **اس‌ام‌اس.آی‌آر**)، API Key، شماره فرستنده، نام قالب‌ها
- API برای درخواست/تأیید OTP (`/api/accounts/otp/request` و `/otp/verify/`)، کد در Redis با اعتبار ۵ دقیقه
- پیامک تأیید سفارش خودکار. تا وقتی پیامک فعال نشده، کد فقط در لاگ ثبت می‌شود

۳. روش‌های ارسال (اسنیپت‌ها «روش‌های ارسال»)
- افزودن/ویرایش هر روش با نام، هزینه، آستانه ارسال رایگان، زمان تحویل
- در صفحه تسویه‌حساب نمایش داده می‌شوند و هزینه ارسال از روی انتخاب کاربر محاسبه می‌شود (دیگر عدد ثابت ۳۰هزار/۵۰۰هزار نیست)

نکات استقرار

:(idempotent) ■■ ■■■■ ■■■■ ■■■ ■■■■■ ■deploy ■■ ■■■

```bash
python manage.py seed_commerce --settings=config.settings.production
```

narevela.ir` ■■■■■■) ■■ ■■■■■ `env.` ■■ ■■ `FRONTEND_URL` ■■■■■■ ■■■■■■ ■■■■ ■

■■■■ ■■■ ■ (`tsc --noEmit` ■ ■manage.py check`■ migration`) ■■ ■■■ ■■■ ■■■
.■■■■■■ ■■■ ■■■■■ API ■■ ■■ ■■■■■■/■■■■ ■■■ .■■ claude/clever-carson-r6f0sg` push`

■■ ■■■■ **■■■■■■ ■■ ■■ ■■■■** ■■ ■■ ■■ ■■■■■■/■■■■ ■■■■ ■■■■■■■ :■■■■ ■■
■(■■■■■ ■■■ ■■■ ■■ ■■■■ login ■■■■ ■■■■■ ■■■■ ■■■■■

کاربر #53

<system-reminder>Message sent at Sun 2026-06-14 18:31:16 UTC.</system-reminder>

الان صفحه وبلاگ رو ساختی؟

اگه نساختی بساز

دستیار (Claude) #54

Let me check the current state of the blog page in the frontend.

دستیار (Claude) #55

No blog page exists. Let me explore how the backend exposes blog data and how the frontend structures other pages.

دستیار (Claude) #56

The backend has a Wagtail `BlogPage` model but `cover_image` and `tags` aren't exposed in the API. Let me look at the frontend patterns and how images/the home blog preview work before building.

دستیار (Claude) #57

Let me look at the products page layout, utils, and the Wagtail blog model section to follow existing patterns.

دستیار (Claude) #58

Let me look at how the homepage renders blocks and fetches data, plus the api blog/image patterns.

دستیار (Claude) #59

Let me check the existing `setup_homepage` command to follow the Wagtail page-tree pattern, and the auth page for client patterns.

دستیار (Claude) #60

I have all the patterns I need. Let me build the blog. First, the backend — expose `cover_image` and tags via the API:

دستیار (Claude) #61

Now add the `ImageRenditionField` import:

دستیار (Claude) #62

Now create a `seed_blog` command so the blog isn't empty after deploy:

دستیار (Claude) #63

Now the frontend. First, add blog types:

دستیار (Claude) #64

Now add the API functions. Let me read the current home-page API section to place them correctly:

دستیار (Claude) #65

Now add the type imports to api.ts:

دستیار (Claude) #66

Now create the blog list page. Let me look at the products page header pattern briefly for consistent styling:

دستیار (Claude) #67

Now create the blog list page:

دستیار (Claude) #68

Now the blog detail page with StreamField body rendering:

دستیار (Claude) #69

Now wire the homepage blog preview to real data. Update `BlogPreviewSection`:

دستیار (Claude) #70

Now update `page.tsx` to fetch blog posts and pass them to both render paths:

دستیار (Claude) #71

Now update the data fetching and both render paths in `HomePage`:

دستیار (Claude) #72

Now the fallback path's `BlogPreviewSection` needs posts passed:

دستیار (Claude) #73

Let me verify the backend has no migration needs (api_fields change is non-DB), run the blog seed, and typecheck the frontend:

دستیار (Claude) #74

Blog seeded. Let me verify the API returns the expected shape via Django shell:

دستیار (Claude) #75

API works correctly (count 4 — one pre-existing post plus my 3 samples). Now the frontend typecheck:

دستیار (Claude) #76

Typecheck passes. Let me verify `next.config` allows the media image host (the detail page cover uses `next/image`):

دستیار (Claude) #77

The config only allows localhost (dev). To ensure blog/product cover images render in production, I'll add the production host to `remotePatterns`:

دستیار (Claude) #78

Now let me run a production build to catch any issues, then commit:

دستیار (Claude) #79

Build succeeds — `/blog` and `/blog/[slug]` are both compiled as dynamic routes. Now let me commit and push:

دستیار (Claude) #80

نه، صفحه وبلاگ ساخته نشده بود — فقط لینک‌هاش توی هدر/فوتر بود ولی خود صفحه `/blog` وجود نداشت (با ۴۰۴ مواجه می‌شدی). الان ساختمش.

چه چیزی اضافه شد

صفحه لیست `/blog`
- نمایش همه مقالات با تصویر شاخص، تاریخ شمسی، نویسنده و برچسب‌ها
- حالت خالی مرتب وقتی مطلبی نیست
- مقالاتی که تصویر ندارند با آیکن جایگزین نمایش داده می‌شوند

صفحه جزئیات `/blog/[slug]`
- نمایش کامل مقاله با رندر بلوک‌های Wagtail StreamField: **عنوان، پاراگراف، تصویر، نقل‌قول**
- تصویر کاور بالای صفحه + متادیتای SEO و OpenGraph

اتصال به دیتابیس (همه از پنل `/cms/` قابل مدیریت)
- مدل `BlogPage`: فیلدهای `cover_image` و `tag_list` به API اضافه شدند (قبلا فقط متن برمی‌گشت)
- توابع `getWagtailImage`، `getBlogPost`، `getBlogPosts` در `lib/api.ts`
- بخش **«آخرین مطالب»** صفحه اصلی هم دیگر داده‌ی ساختگی نیست و مقالات واقعی را نشان می‌دهد (اگر مقاله‌ای نباشد، بخش پنهان می‌شود)

دستور seed
- `seed_blog` فهرست مقالات و ۳ مقاله نمونه می‌سازد (idempotent، با `--force` برای بازنویسی)

■■■■■■ ##
■■ api_fields ■■■■■■ — ■■■■ migration ■■■■) `makemigrations --check` ■ `manage.py check` ■ -
(■■■■■■ migration
■■■■■■■■■■■ ■■■■ ■■■■■■■ ■ ■■ ■■■ dev ■■■ API ■ -
■■■■ ■■■■■ `[blog/[slug/` ■ `blog/` ■■■■■■■■ — ■■■■ `npm run build` ■ `tsc --noEmit` ■ -
■■ claude/clever-carson-r6f0sg` push` ■■■■ ■■■ ■■■ -

deploy ■■ ■■■ ■■■■ ■■■ ##
:■■ ■■■■ ■■■■■■ ■■■■■ ■■■ ■■ ■■■■■

```bash
python manage.py seed_blog --settings=config.settings.production
```

■■ ■■■■ ■■■■■■ ■■■■■■■■ ■■■■ ■■■■ ■**■■■■■■** ■■■ `/cms/` ■■■ ■■ ■■■■■■■ ■■■■■
.■■■■■■ ■■■■■ ■■■ — ■■■ ■■■■■■ ■■■■■■

■■ ■■■■■■ ■■ ■■ (tag ■■■ ■■■■■) **■■■■■■** ■■■■ ■■ ■■■■■■ ■■■■■■ ■■■■■■ ■■■■■■■
■■■■■■■

کاربر #81

<system-reminder>Message sent at Sun 2026-06-14 18:41:55 UTC.</system-reminder>
کل پیام هایی که رد و بدل کردیم رو در یک فایل پی دی اف ذخیره کن

دستیار (Claude) #82

I'll save our full conversation to a PDF. Persian text needs proper RTL rendering, so let me first locate the transcript and check what PDF tools and fonts are available.

دستیار (Claude) #83

pip works (network available), reportlab + arabic_reshaper + python-bidi installed. Let me grab a proper Persian font (Vazirmatn):

دستیار (Claude) #84

Got the Vazirmatn font. Now let me inspect the transcript structure to parse messages correctly:

دستیار (Claude) #85

Let me extract the actual conversational messages and preview them before building the PDF:

دستیار (Claude) #86

Font and parser ready. Now I'll write a PDF generator using reportlab with Arabic reshaping + bidi, registering Vazirmatn, and per-line direction detection so Persian renders RTL and code/paths stay LTR: